

Введение

Вы - ученый-гражданин, который начал сбор данных о подъеме воды в реке рядом с тем местом, где вы живете. В течение нескольких месяцев вы кропотливо измеряете уровень воды и записываете свои выводы в блокнот. Но в конце концов, что именно у вас есть? Что могут сказать нам все эти данные?

В этом уроке мы рассмотрим, как мы можем использовать NumPy для анализа данных. Мы научимся различным методам расчета общих статистических свойств набора данных, таких как определение среднего и стандартного отклонения. К концу вы сможете провести базовый анализ набора данных и понять, как мы можем использовать статистику, чтобы делать выводы о данных.

Мы рассмотрим следующие статистические концепции:

- Среднее
- Медиана
- Процентили
- Межквартильный размах
- Выбросы
- Среднеквадратичное отклонение

Для начала мы проанализируем наборы данных с одной переменной. Один из способов представить себе набор данных с одной переменной — это то, что он содержит ответы на вопрос. Например, мы можем спросить 100 человек: «Какой у вас рост?» Их высота в дюймах будет составлять наш набор данных.

Для наших целей мы будем организовывать наши наборы данных в массивы NumPy.

Задание

Ознакомиться с примером:

После того, как река в вашем городе затопила во время недавнего урагана, вы заинтересовались сбором данных о ее высоте. Каждый день в течение последнего месяца вы идете к реке, измеряете высоту воды и записываете эту информацию в блокнот.

Давайте посмотрим, как вы можете использовать функции NumPy для анализа вашего набора данных.

Сначала мы импортируем модуль NumPy, чтобы мы могли использовать его функции статистических вычислений.

```
import numpy as np

water_height = np.array([4.01, 4.03, 4.27, 4.29, 4.19,
                        4.15, 4.16, 4.23, 4.29, 4.19,
                        4.00, 4.22, 4.25, 4.19, 4.10,
                        4.14, 4.03, 4.23, 4.08, 14.20,
                        14.03, 11.20, 8.19, 6.18, 4.04,
                        4.08, 4.11, 4.23, 3.99, 4.23])
```

Давайте воспользуемся функцией `np.mean()`, чтобы найти среднюю высоту воды:

```
In [11]: np.mean(water_height)
Out[11]: 5.2510000000000003
```

Но подождите! Мы должны отсортировать наши данные, чтобы увидеть, могут ли быть какие-либо измерения, которые могут сильно отклоняться от среднего:

```
In [12]: np.sort(water_height)
Out[12]: array([ 3.99,  4.   ,  4.01,  4.03,  4.03,  4.04,  4.08,  4.08,
                 4.1   ,  4.11,  4.14,  4.15,  4.16,  4.19,  4.19,  4.19,
                 4.22,  4.23,  4.23,  4.23,  4.23,  4.25,  4.27,  4.29,
                 4.29,  6.18,  8.19, 11.2  , 14.03, 14.2  ])
```

Похоже, эта гроза могла повлиять на среднюю высоту! Давайте измерим медиану, чтобы увидеть, является ли она более репрезентативной (наглядной) для исследуемого набора данных:

```
In [13]: np.median(water_height)
Out[13]: 4.1900000000000004
```

Хотя медиана говорит нам, где находится середина наших данных, давайте посмотрим на значение ближе к концу набора данных. Мы можем использовать процентиля, чтобы использовать позицию точек данных и получить ее значение:

```
In [14]: np.percentile(water_height, 75)
Out[14]: 4.2649999999999997
```

Мы получили представление о значениях нашего набора данных. Но как насчет распространения наших данных? Давайте рассчитаем стандартное отклонение, чтобы понять, насколько похожи или насколько различны каждая из точек данных:

```
In [15]: np.std(water_height)
Out[15]: 2.784585367099861
```

Просто используя несколько простых функций, мы смогли быстро вычислить несколько важных измерений и приступить к анализу нашего набора данных.

NumPy и Mean

Первое статистическое понятие, которое мы рассмотрим, — это среднее значение, также обычно называемое средним. Среднее значение - полезное измерение для определения

центра набора данных. NumPy имеет встроенную функцию для вычисления среднего или среднего значений массивов: `np.mean`

Предположим, мы хотим найти среднее количество фунтов продукции, которое человек покупает в неделю. Мы провели опрос и получили 1000 ответов:

```
survey_responses = [5, 10.2, 4, .3 ... 6.6]
```

Затем мы можем преобразовать набор данных в массив NumPy и использовать функцию `np.mean` для вычисления среднего значения:

```
>>> survey_array = np.array(survey_responses)
>>> np.mean(survey_array)
5.220
```

Задание

1. Дистрибьютор напитков Wine Not? хочет подсчитать, сколько бутылок Bourdeaux он продает в среднем у трех местных продавцов вина, чтобы определить, следует ли увеличивать свои запасы. Продажи за последнюю неделю для каждого магазина перечислены ниже.

```
store_one = np.array([2, 5, 8, 3, 4, 10, 15, 5])
store_two = np.array([3, 17, 18, 9, 2, 14, 10])
store_three = np.array([7, 5, 4, 3, 2, 7, 7])
```

Найдите средние продажи для каждого магазина и сохраните их в переменных `store_one_avg`, `store_two_avg` и `store_three_avg`.

2. Выведите в консоль все полученные средние
3. Обратите внимание на средние продажи в неделю для каждого магазина. Начальник говорит, что мы должны увеличивать запасы, но только в том случае, если средние продажи в магазине превышают 7 бутылок в неделю. Сохраните имя переменной набора данных магазина, которое соответствует этому описанию, в переменной `best_seller`.

Средние и логические операции

Мы также можем использовать `np.mean` для вычисления процента элементов массива, у которых есть определенное свойство.

Как мы знаем, логический оператор будет оценивать каждый элемент в массиве, чтобы убедиться, что он соответствует указанному условию. Если элемент соответствует заданному условию, он будет оцениваться как `True` и равен 1. Если он не соответствует, он будет `False` и равен 0.

Когда `np.mean` вычисляет логический оператор, результирующее среднее значение будет эквивалентно общему количеству элементов `True`, разделенному на общую длину массива.

В нашем примере опроса продукции мы можем использовать этот расчет, чтобы узнать процент людей, которые покупали более 8 фунтов продукции каждую неделю:

```
>>> np.mean(survey_array > 8)
0.2
```

Логический оператор `survey_array > 8` оценивает, какие ответы на опрос были больше 8, и присваивает им значение 1. `np.mean` складывает все единицы и делит их на длину массива `survey_array`. Полученные результаты говорят нам, что 20% респондентов купили более 8 фунтов продукции.

Задание

1. Вы проводите встречу выпускников местного колледжа. У вас есть список с именами всех присутствующих и годом, когда они закончили обучение.

Мы сохранили этот список как массив NumPy в переменной `class_year`. Подсчитайте процент выпускников, получивших высшее образование в 2005 году и позже, и сохраните свой результат в переменной `миллениалы`.

```
class_year = np.array([1967, 1949, 2004, 1997, 1953, 1950, 1958, 1974, 1987, 2006,
, 2013, 1978, 1951, 1998, 1996, 1952, 2005, 2007, 2003, 1955, 1963, 1978, 2001, 2
012, 2014, 1948, 1970, 2011, 1962, 1966, 1978, 1988, 2006, 1971, 1994, 1978, 1977
, 1960, 2008, 1965, 1990, 2011, 1962, 1995, 2004, 1991, 1952, 2013, 1983, 1955, 1
957, 1947, 1994, 1978, 1957, 2016, 1969, 1996, 1958, 1994, 1958, 2008, 1988, 1977
, 1991, 1997, 2009, 1976, 1999, 1975, 1949, 1985, 2001, 1952, 1953, 1949, 2015, 2
006, 1996, 2015, 2009, 1949, 2004, 2010, 2011, 2001, 1998, 1967, 1994, 1966, 1994
, 1986, 1963, 1954, 1963, 1987, 1992, 2008, 1979, 1987])
```

2. Выведите в консоль результат

Вычисление среднего двумерных массивов

Если у нас есть двумерный массив, `np.mean` может вычислить средние значения `dstuj` большего массива, а также внутренних значения (подмассивов).

Представим себе игру в бросок кольца на карнавале. В этой игре у вас есть три разных шанса собрать все три кольца на палку. В нашем массиве `ring_toss` каждый внутренний массив (массивы в более крупном массиве) — это одна попытка, а каждое число - это один бросок кольца. 1 представляет успешный бросок, 0 представляет неудачу.

Во-первых, мы можем использовать `np.mean`, чтобы найти среднее значение по всем массивам:

```
>>> ring_toss = np.array([[1, 0, 0],
, [0, 0, 1],
, [1, 0, 1]])
```

```
>>> np.mean(ring_toss)
0.44444444444444442
```

Чтобы найти среднее каждого внутреннего массива, мы указываем ось 1 («строки»):

```
>>> np.mean(ring_toss, axis=1)
array([ 0.33333333,  0.33333333,  0.66666667])
```

Чтобы найти среднее значение каждой позиции индекса (т.е. среднее значение всех первых бросков, среднее значение всех вторых бросков,...), мы указываем ось 0 («столбцы»):

```
>>> np.mean(ring_toss, axis=0)
array([ 0.66666667,  0.          ,  0.66666667])
```

Задание

1. Мы предоставили данные об испытании нового лекарства от аллергии AllerGeeThatSucks! Пятерых участников попросили оценить, насколько сонливыми они были в результате приема лекарства, раз в день в течение трех дней по шкале от одного (наименьшая сонливость) до десяти (наибольшая сонливость).

Используйте `np.mean`, чтобы найти средний уровень сонливости по всем испытаниям и сохранить результат в переменной `total_mean`.

```
import numpy as np

allergy_trials = np.array([[6, 1, 3, 8, 2],
                           [2, 6, 3, 9, 8],
                           [5, 2, 6, 9, 9]])
```

2. Используйте `np.mean`, чтобы найти средний уровень сонливости за каждый день эксперимента, и сохраните его в переменной `trial_mean`.
3. Используйте `np.mean`, чтобы найти средний уровень сонливости для каждого отдельного пациента, чтобы увидеть, были ли некоторые из них более чувствительны к препарату, чем другие, и сохраните его в переменной `Patient_mean`.
4. Выведите в консоль все значения средних

Выбросы

Как мы видим, среднее значение - полезный способ быстро понять различные части наших данных. Однако на среднее значение сильно влияют конкретные значения в нашем наборе данных. Что произойдет, если одно из этих значений значительно отличается от остальных?

Значения, не попадающие в большую часть набора данных, называются выбросами. Важно идентифицировать выбросы, потому что, если они останутся незамеченными, они могут исказить наши данные и привести к ошибке в нашем анализе (например, при определении среднего значения). Они также могут быть полезны для выявления ошибок в нашем сборе данных.

Когда мы сможем идентифицировать выбросы, мы сможем определить, были ли они вызваны ошибкой при сборе выборки или представляют собой значительное, но реальное отклонение от среднего значения.

Предположим, мы хотим определить средний рост учеников 3-го класса. Мы измеряем нескольких учеников в местной школе, но случайно измеряем одного ученика в сантиметрах, а не в дюймах. Если мы не будем обращать на это внимание, наш набор данных может выглядеть так:

```
[50, 50, 51, 49, 48, 127]
```

В этом случае 127 будет выбросом.

Некоторые выбросы не являются результатом ошибки. Например, предположим, что один из наших третьеклассников пропустил класс и был на год младше всех остальных в классе:

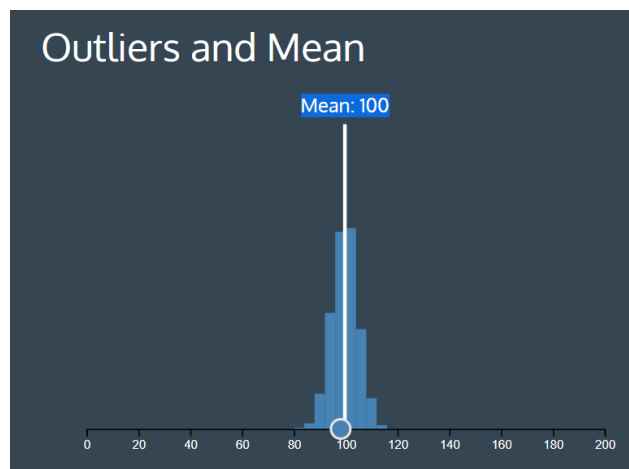
```
[50, 50, 51, 49, 48, 45]
```

Она могла бы быть значительно ниже (45 дюймов), но ее рост все равно был бы исключением.

Предположим, что другой ученик был просто необычно высоким для своего возраста:

```
[50, 50, 51, 49, 48, 58.5]
```

Его рост 58,5 дюймов также был бы исключением.



Сортировка и выбросы

Один из способов быстро выявить выбросы — это отсортировать наши данные. После сортировки данных мы можем быстро взглянуть на начало или конец массива, чтобы увидеть, не лежат ли некоторые значения далеко за пределами ожидаемого диапазона. Мы можем использовать функцию NumPy `np.sort` для сортировки наших данных.

Вернемся к нашему примеру с ростом в третьем классе и представим, что восьмиклассник вошел в наш эксперимент:

```
>>> heights = np.array([49.7, 46.9, 62, 47.2, 47, 48.3, 48.7])
```

Если мы воспользуемся `np.sort`, мы сразу сможем идентифицировать более высокого ученика, поскольку его рост (62 дюйма) заметно выходит за пределы диапазона набора данных:

```
>>> np.sort(heights)
array([ 46.9,  47. ,  47.2,  48.3,  48.7,  49.7,  62])
```

Задание

1. Вы отслеживали данные о температуре летом на своем заднем крыльце, но поняли, что разместили датчик прямо над решеткой! Прежде чем вы сможете использовать свои данные, вам нужно проверить, не вызвало ли тепло от гриля каких-либо странных показаний, которые могли бы исказить ваши данные.

Сначала отсортируйте массив данных `temps` и сохраните отсортированные данные в переменной `sorted_temps`.

```
temps = np.array([86, 88, 94, 85, 97, 90, 87, 85, 94, 93, 92, 95, 98, 85, 94, 91,
97, 88, 87, 86, 99, 89, 89, 99, 88, 96, 93, 96, 85, 88, 191, 95, 96, 87, 99, 93,
90, 86, 87, 100, 187, 98, 101, 101, 96, 94, 96, 87, 86, 92, 98, 94, 98, 90, 99, 9
6, 99, 86, 97, 98, 86, 90, 86, 94, 91, 88, 196, 195, 93, 97, 199, 87, 87, 90, 90,
98, 88, 92, 97, 88, 85, 94, 88, 93, 198, 90, 91, 90, 92, 92])
```

2. Вывести в консоль результат

NumPy и медиана

Еще один ключевой показатель, который мы можем использовать при анализе данных, — это медиана. Медиана — это среднее значение набора данных, упорядоченного по величине (от наименьшего к наибольшему).

Давайте посмотрим на следующий массив:

```
np.array([1, 1, 2, 3, 4, 5, 5])
```

В этом примере медиана будет равна 3, потому что она расположена посередине между минимальным и максимальным значениями.

Если бы длина нашего набора данных была четным числом, медиана была бы значением посередине между двумя центральными значениями. Итак, в следующем примере медиана будет равна 3,5:

```
np.array([1, 1, 2, 3, 4, 5, 5, 6])
```

В

Но что, если бы у нас был очень большой набор данных? Было бы очень утомительно подсчитывать все значения. К счастью, в NumPy также есть функция для вычисления медианы `np.median`:

```
>>> my_array = np.array([50, 38, 291, 59, 14])
>>> np.median(my_array)
50.0
```

Задание

1. Вы исследуете семейные доходы и наткнулись на следующий небольшой набор данных:

```
10100, 35500, 105000, 85000, 25500, 40500, 65000
```

Вычислите медиану, не используя NumPy, и сохраните значение в переменной `small_set_median`.

2. По мере того, как вы продолжаете свое исследование, вы обнаруживаете кладезь исследований в файле `home_income.csv` (данные ниже), который необходимо поместить в переменную `large_set`.

Используйте NumPy, чтобы найти медиану `large_set` и сохранить результат в переменной `large_set_median`.

```
173306.  102204.  96767.  54264.  40898.  67887.
118737.   74847.
  104813.  122787.  122577.  113890.  100576.  90345.
94656.   71403.
  153102.  228233.  85628.  106708.  57289.  42279.
157273.   71691.
   77030.  111763.  30770.  69563.  126201.  114726.
78125.   42670.
  161304.   74165.  114605.  27270.   3283.  84450.
106449.   89038.
   91897.   50975.  133062.  151264.  66383.  90675.
70498.   36092.
  160018.   58767.  121995.  63990.  226476.  111729.
127747.   57527.
```



```

205382. 117022. 147723. 71284. 119317. 138942.
62945. 121499.
66940. 80725. 47357. 98913. 79435. 92556.
151156. 126723.
65019. 187295. 47038. 40008. 79243. 45334.
142448. 63090.
92153. 251255. 50875. 46395. 76754. 43275.
43150. 57908.
89343. 39451. 43198. 89666. 55168. 41133.
185956. 239637.
63426. 153088. 86450. 195881. 78946. 80067.
69686. 184432.
154515. 115804. 133530. 60730. 44099. 67570.
62957. 140604.
42206. 94150. 86555. 152236. 91549. 105412.
82216. 117358.
161862. 101036. 117315. 104048. 83689. 65307.
71789. 67782.
61844. 71737. 203751. 99121. 44308. 47353.
44960. 82439.
96041. 60345. 189146. 140438. 162413. 94489.
173568. 60460.
154057. 137258. 151520. 77844. 218342. 132205.
60806. 58737.
184590. 95922. 49074. 119246. 126128. 35538.
143795. 59662.
65359. 187713. 128584. 134499. 166816. 105213.
76383. 29964.
73273. 73307. 110846. 80573. 68186. 52601.
27428. 32568.
178872. 111952. 42409. 188881. 132103. 53882.
41144. 60130.
93221. 132447. 143502. 110259. 55159. 44512.
46611. 55662.
142456. 70376. 67326. 53772. 75131. 36100.
121742. 33156.

```

3. Вывести полученные результаты в консоль

Среднее против медианы

В наборе данных значение медианы может обеспечить важное сравнение со средним значением. В отличие от среднего, на медианное значение не влияют выбросы. Это становится важным в наборах данных с перекосом, в наборах данных, значения которых распределяются неравномерно. Давайте напишем программу, которая исследует эту идею дальше.

Задание

1. Компания, в которой вы работаете, хочет знать среднее количество времени, которое кто-то проводит на веб-сайте компании в минутах. На основании данных ниже создайте файл CSV. Импортируйте этот набор данных в вашу программу как `time_spent`.

```
2. [ 0.  0.  0.  0.  0.  0.  0.  0.  0.
    0.  0.  0.  0.
3.  0.  0.  0.  0.  0.  0.  0.  0.  0.
    0.  0.  0.  0.
4.  0.  0.  0.  0.  0.  0.  0.  0.  0.
    0.  1.  1.  1.
5.  1.  1.  1.  1.  1.  1.  1.  1.  1.
    1.  1.  1.  1.
6.  1.  1.  1.  1.  1.  1.  1.  1.  1.
    1.  1.  1.  1.
7.  1.  1.  1.  1.  1.  1.  1.  1.  1.
    1.  1.  1.  1.
8.  1.  1.  1.  1.  1.  1.  1.  1.  1.
    1.  1.  1.  1.
9.  1.  1.  1.  1.  1.  1.  1.  2.  2.
    2.  2.  2.  2.
10.  2.  2.  2.  2.  2.  2.  2.  2.  2.
    2.  2.  2.  2.
11.  2.  2.  2.  2.  2.  2.  2.  2.  2.
    2.  2.  2.  3.
12.  3.  3.  3.  3.  3.  3.  3.  3.  3.
    3.  3.  3.  3.
13.  3.  3.  3.  3.  3.  3.  3.  3.  3.
    3.  3.  3.  3.
14.  3.  3.  3.  3.  3.  3.  3.  3.  4.
    4.  4.  4.  4.
15.  4.  4.  4.  4.  4.  4.  4.  4.  4.
    4.  4.  4.  4.
16.  4.  4.  4.  4.  4.  4.  4.  4.  5.
    5.  5.  5.  5.
17.  5.  5.  5.  5.  5.  5.  5.  5.  5.
    5.  5.  5.  5.
18.  5.  6.  6.  6.  6.  6.  9.  10.  22.
    60. 60. 61. 72.
19. 75. 82. 103. 120. 210. 304.]
```

3. Найдите среднее количество времени в минутах, проведенных на веб-сайте, и сохраните его в переменной `minutes_mean`.
4. Теперь найдите медианное значение массива и сохраните его в переменной `minutes_median`.

5. Теперь найдите медианное значение массива и сохраните его в переменной `minutes_median`.
6. Выведите значение среднего и медианы в консоль
7. Обратите внимания на разницу между двумя значениями. Как вы думаете, какой из них лучше всего отражает большинство имеющихся данных?
Выберите переменную, представляющую это значение, и сохраните ее в переменной `best_measure`.

Процентили, часть I

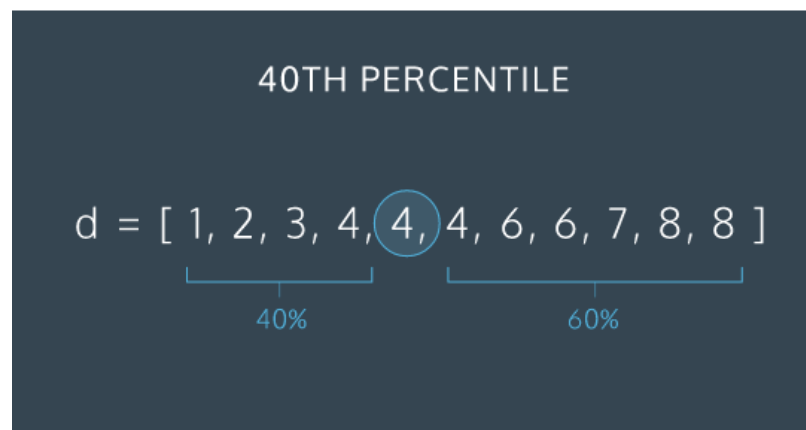
Как мы знаем, медиана — это середина набора данных: это число, для которого 50% выборок находятся ниже, а 50% выборок - выше. Но что, если бы мы хотели найти точку, в которой 40% образцов находятся ниже, а 60% образцов находятся выше?

Этот тип точки называется процентилем. N-й процентиль определяется как точка N% выборок, лежащих ниже него. Таким образом, точка, в которой ниже 40% выборок, называется 40-м процентилем. Процентили - полезные измерения, потому что они могут сказать нам, где находится конкретное значение в более крупном наборе данных.

Давайте посмотрим на следующий массив:

```
d = [1, 2, 3, 4, 4, 4, 6, 6, 7, 8, 8]
```

В наборе данных 11 чисел. 40-й процентиль будет иметь 40% из 10 оставшихся чисел под ним (40% от 10 равно 4) и 60% чисел над ним (60% от 10 равно 6). Итак, в этом примере 40-й процентиль равен 4.



В NumPy мы можем вычислять процентили, используя функцию `np.percentile`, которая принимает два аргумента: массив и процентиль для вычисления.

Вот как мы могли бы использовать NumPy для вычисления 40-го перцентиля массива `d`:

```
>>> d = np.array([1, 2, 3, 4, 4, 4, 6, 6, 7, 8, 8])
>>> np.percentile(d, 40)
4.00
```

Задание

1. Местная публичная библиотека хочет узнать, сколько часов в неделю их посетители используют компьютеры. Вверху script.py мы включили образцы данных от 11 пользователей в массив NumPy.

```
patrons = np.array([ 2, 6, 14, 4, 3, 9, 1, 11, 4, 2, 8])
```

Используйте NumPy, чтобы найти 30-й процентиль отсортированного массива и сохранить его в переменной с именем `thirtieth_percentile`.

2. Затем используйте NumPy, чтобы найти 70-й процентиль и сохранить его в переменной `seventieth_percentile`.
3. Выведите в консоль результаты расчета

Процентили, часть II

Некоторые процентили имеют определенные имена:

- 25-й процентиль называется первым квартилем.
- 50-й процентиль называется медианой.
- 75-й процентиль называется третьим квартилем.

Минимум, первый квартиль, медиана, третий квартиль и максимум набора данных называются сводкой из пяти чисел. Этот набор чисел отлично подходит для вычисления, когда мы получаем новый набор данных.

Разница между первым и третьим квартилями — это величина, называемая межквартильным размахом. Например, допустим, у нас есть следующий массив:

```
d = [1, 2, 3, 4, 4, 4, 6, 6, 7, 8, 8]
```

Мы можем вычислить 25-й и 75-й процентили, используя `np.percentile`:

```
np.percentile(d, 25)
>>> 3.5
np.percentile(d, 75)
>>> 6.5
```

Затем, чтобы найти межквартильный размах, мы вычитаем значение 25-го перцентиля из значения 75-го:

$$6.5 - 3.5 = 3$$

50% набора данных будет находиться в пределах межквартильного диапазона. Межквартильный размах дает нам представление о разбросе наших данных. Чем меньше значение межквартильного размаха, тем меньше разброс в нашем наборе данных. Чем больше значение, тем больше дисперсия.

Задание

1. Компания стриминговый сервис фильмов хочет знать, сколько фильмов пользователи смотрят за одну неделю. Мы включили образцы данных от 15 пользователей в массив.

```
movies_watched = np.array([2, 3, 8, 0, 2, 4, 3, 1, 1, 0, 5, 1, 1, 7, 2])
```

Найдите 25-й и 75-й процентиля и сохраните их в соответствующих переменных: `first_quarter` и `3rd_quarter`.

2. Создайте переменную с именем `interquartile_range`. Вычислите межквартильный размах и сохраните его в этой переменной.
3. Вывести в консоль результаты

NumPy и стандартное отклонение, часть I

Хотя среднее значение и медиана могут сказать нам о центре наших данных, они не отражают диапазон данных. Вот тут и появляется стандартное отклонение.

Подобно межквартильному размаху, стандартное отклонение говорит нам о разбросе данных. Чем больше стандартное отклонение, тем больше наши данные разбросаны от центра. Чем меньше стандартное отклонение, тем больше данные сгруппированы вокруг среднего значения.

NumPy и стандартное отклонение, часть II

Как мы видели в последнем упражнении, знание стандартного отклонения набора данных может помочь нам понять, насколько разбросан наш набор данных.

Мы можем найти стандартное отклонение набора данных с помощью функции Numpy `np.std`:

```
>>> nums = np.array([65, 36, 52, 91, 63, 79])
>>> np.std(nums)
17.716909687891082
```

Задание

1. Вас попросили оценить ежегодный фестиваль сквоша в вашем городе. Организатор фестиваля дает вам список, в котором указаны все веса для двух соревнований, которые вы судите: тыквы и желудевые сквоши.

Учитывая два набора данных, найдите средний вес для каждого соревнования и сохраните его в переменных `pumpkin_avg` и `acorn_squash_avg`.

```
pumpkin = np.array([68, 1820, 1420, 2062, 704, 1156, 1857, 1755, 2092, 1384])  
  
acorn_squash = np.array([20, 43, 99, 200, 12, 250, 58, 120, 230, 215])
```

2. Теперь остальные судьи хотят, чтобы вы дали им представление о том, насколько репрезентативны средние значения по отношению к полноте представленных материалов. Вычислите стандартное отклонение для каждого набора данных, чтобы найти и сохранить их в переменных `pumpkin_std` и `acorn_squash_std`. Выведите их на консоль, чтобы увидеть их значения.
3. Определите набор данных сквоша, который имеет большее стандартное отклонение, и сохраните его для переменной `winner`.

Заключение

Давайте рассмотрим! В этом уроке вы узнали, как использовать NumPy для анализа наборов данных с одной переменной. Вот что мы рассмотрели:

- Использование метода `np.sort` для поиска выбросов.
- Вычисление центральных позиций набора данных с использованием `np.mean` и `np.median`.
- Понимание разброса наших данных с использованием процентилей и межквартильного размаха.
- Нахождение стандартного отклонения набора данных с помощью `np.std`.

Задание

1. Группа ученых собирала данные об осадках в Сиэтле. Они представили вам свои данные в виде среднемесячных значений, измеряемых в дюймах.

Month	Avg. Precipitation
January	5.21
February	3.76
March	3.27
April	2.35
May	1.89
June	1.55
July	0.65
August	1.06

September	1.72
October	3.36
November	4.82
December	5.11

Сохраните эти данные в массиве NumPy `rainfall`.

2. Найдите среднее значение массива осадков и сохраните его в переменной `rain_mean`.
3. Найдите медианное значение массива осадков и сохраните его в переменной `rain_median`.
4. Найдите 25-й и 75-й проценти́ли исходного массива осадков и сохраните их в массивах `first_quarter` и `third_quarter` соответственно.
5. Вычислите межквартильный размах и сохраните его в переменном межквартильном размахе.
6. Определите стандартное отклонение набора и сохраните его в переменной `rain_std`.
7. Вывести результаты в консоль